

EXTERNAL INTEGRATION

MyScoutee Integration API Guide

A compact API for asset and event batches, organizer event links, participant invitations, completed Speed meeting encounters, and scoped change callbacks.

DOCUMENT ID MSC-API-001	DOCUMENT VERSION 1.0.1
APPLICABLE SOFTWARE VERSION MyScoutee 1.0.1	STATUS ● Released
RELEASE DATE 2026-09-20	SOURCE REVISION 2026-09-21
LANGUAGE EN	

Document Control

Document status and applicability

Document owner	MyScoutee project — integration documentation
Intended audience	External system developers, integration operators, and maintainers of the MyScoutee Java client
Normative API prefix	The deployment-specific Base URL shown in the profile API popup; its usual path is <code>/api/integrations/v1</code>
Reference implementation	<code>myscoutee-client</code> version 1.0.1, Java 17

Version History

Version	Date	Status	Change
1.0.1	2026-09-20	Released	Initial public batch API, scoped callback watch, and Java client guide.
1.0.1 + source extensions	2026-09-21	Implementation revision	Organizer event links, Speed meeting configuration, participant links, event reads and completed encounter export/watch. New commands require a client built from this source revision.
MSC-116 source extension	2026-10-06	Implementation revision; execution QA pending	Three access packs, application reads and organizing commands. Java client 1.5.0 and MCP 1.5.0. No public plugin release is implied.

Small by design

The token fixes the owner for batch writes, managed-event reads, participant invitation links and encounter exports. This guide includes the current source extensions; their presence in a deployment depends on its installed server and client revision.

Table of Contents

Integration Model

Base URL and Authentication

Endpoint Reference

ID Correlation and Retry Behaviour

Asset Batch

Event Batch

Optional Image Upload

Change Callback

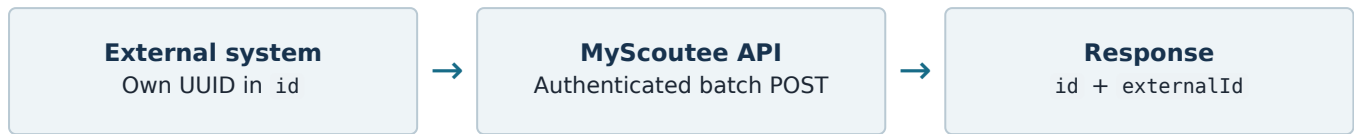
Responses and Errors

Java Client and CLI

Security and Operational Rules

MSC-116: Access Packs and Application Queries

1. Integration Model



Concept	Meaning
<code>id</code>	The caller-owned UUID sent in the request. It identifies the source-system record.
<code>externalId</code>	The stable MyScoutee asset or event UUID returned for that request item.
Ownership	Every created resource belongs only to the user who owns the integration token.
Batch	One request contains 1-1000 items. Further batches may be sent later.
Image	Optional local file bytes sent with the same POST. No remote image URL is accepted or downloaded.

2. Base URL and Authentication

Connect one external client

- 1 Open the API popup from the lower-right corner of the profile identity header.
- 2 Copy the full Base URL, including scheme, domain, and `/api/integrations/v1` path.
- 3 Generate an expiring client key. The complete token is shown only once.
- 4 Configure one stable client UUID and call `POST /connect`.

Header	Value	Required
<code>Authorization</code>	<code>Bearer msc_...</code>	Yes
<code>X-MyScoutee-Client-Id</code>	A stable UUID generated once by the external client	Yes
<code>Accept</code>	<code>application/json</code>	Recommended
<code>Content-Type</code>	<code>application/json</code> or <code>multipart/form-data</code>	Yes for batch POSTs

One key, one client

The first successful client UUID claims the key. Another client UUID cannot use it. A user may keep at most three active keys, and revocation blocks a key immediately.

3. Endpoint Reference

Method	Path	Purpose	Success
POST	/connect	Claim and validate a client key	200
POST	/assets	Create 1-1000 assets, optionally with images	200
POST	/events	Create 1-1000 events, optionally with images	200
GET	/events/{externalId}	Read a managed event, including its external link and format	200
POST	/events/{externalId}/invites	Create links for 1-1000 participant IDs	200
GET	/events/{externalId}/encounters	Read completed, unique encounter pairs; offset/limit pagination	200
POST	/watch	Register or disable this client's scoped callback	200

There is no separate image endpoint and no booking query endpoint in this API version.

4. ID Correlation and Retry Behaviour

```
{
  "items": [
    {
      "id": "1066b61b-c0ac-43d5-ab28-f588f5a162f6",
      "externalId": "55fe8563-f360-48cf-af0e-7cfb0404ee84",
      "result": "created",
      "error": null
    }
  ]
}
```

The caller stores the pair above. Repeating the same resource type and caller `id` for the same token owner returns the same `externalId` with `result: "existing"`. It does not create or update a second resource.

Safe request retry

If a client times out before receiving the response, it may repeat the POST with the same item IDs and recover the same MyScoutee IDs.

5. Asset Batch

Field	Type	Required	Rule
id	UUID	Yes	Unique within the batch and stable in the caller system
type	string	Yes	Transport, Accommodation, or Supplies
title	string	Yes	Non-blank
capacityTotal	integer	Yes	At least 1
quantity	integer	Yes	At least 1
routes	string array	Accommodation only	At least one non-blank location for accommodation
subtitle, category, city, details	string	No	Asset description fields
sourceLink	URI string	No	Optional source reference; it is not used to download an image
visibility	string	No	Defaults through the ordinary asset model when omitted

```
{
  "items": [{
    "id": "7d857d7f-8d48-4cf1-95cc-cbf257640afa",
    "type": "Accommodation",
    "title": "Hotel room 101",
    "capacityTotal": 2,
    "quantity": 1,
    "routes": ["Bratislava"],
    "visibility": "Private"
  }]
}
```

6. Event Batch

Field	Type	Required	Rule
<code>id</code>	UUID	Yes	Unique within the batch and stable in the caller system
<code>title</code>	string	Yes	Non-blank
<code>subtitle</code>	string	Yes	Non-blank, matching the event editor requirement
<code>startAt</code> , <code>endAt</code>	ISO-8601 instant	Yes	<code>endAt</code> must be after <code>startAt</code>
<code>capacityMin</code>	integer	Yes	Zero or greater
<code>capacityMax</code>	integer	Yes	At least <code>capacityMin</code>
<code>location</code>	string	No	Optional location text
<code>sourceLink</code>	string	No	External organizer event-page link, following the asset link model. Returned in event reads and change callbacks; not a separate payment URL.
<code>mode</code>	string	No	<code>Casual</code> (default), <code>Tournament</code> or <code>Mingle</code> . UI labels are Standard, Tournament and Speed meeting.
<code>mingleConfiguration</code>	object	No	Allowed only with <code>mode: Mingle</code> . If present, all five fields below are required. No configuration is invented when omitted.
<code>visibility</code>	string	No	Ordinary event visibility; defaults when omitted
<code>status</code>	string	No	<code>draft</code> or <code>active</code> ; defaults to <code>active</code>
<code>topics</code>	string array	No	Optional event topics

```
{
  "items": [{
    "id": "1066b61b-c0ac-43d5-ab28-f588f5a162f6",
    "title": "Hotel evening",
    "subtitle": "Welcome event for hotel guests",
    "startAt": "2026-10-10T18:00:00Z",
    "endAt": "2026-10-10T20:00:00Z",
    "capacityMin": 1,
    "capacityMax": 30,
    "status": "draft",
    "sourceLink": "https://organizer.example.test/events/hotel-evening",
    "mode": "Mingle",
    "mingleConfiguration": {
```

```

    "groupSize": 4,
    "plannedRounds": 3,
    "roundDurationMinutes": 10,
    "breakDurationMinutes": 2,
    "requireGenderBalance": false
  }
}]
}
```

Configuration limits: `groupSize` 2-20, `plannedRounds` 1-100, `roundDurationMinutes` 1-240, `breakDurationMinutes` 0-60 and boolean `requireGenderBalance`. The app displays Speed meeting (Hungarian: Villámtalálkozók); the wire identifier stays `Mingle`.

6.1 Read a managed event

GET `/events/{externalId}` returns `id`, `externalId`, `title`, `status`, `sourceLink`, `mode` and `mingleConfiguration`. Use the MyScoutee event ID in the path. The caller-owned `id` can be null for events created in the app. Another owner's event returns 404.

6.2 Create participant links

POST `/events/{externalId}/invites` accepts 1-1000 distinct caller-owned participant IDs. IDs are trimmed, nonblank strings of at most 200 characters; they need not be UUIDs.

```

{"participantIds": ["guest-001", "guest-002"]}
```

```

{
  "items": [{
    "id": "guest-001",
    "inviteUrl": "https://example.com/game?partnerInvite=opaque-private-token",
    "claimed": false
  }, {
    "id": "guest-002",
    "inviteUrl": "https://example.com/game?partnerInvite=another-private-token",
    "claimed": false
  }]
}
```

Repeating the same owner/event/participant tuple returns the same link. Send each private link to its intended participant. The link survives sign-in or registration and binds the external participant ID to the authenticated account. These links have no time-based expiry. A claimed link cannot be transferred to a different account; event ownership and participant-account conflicts are still checked.

Before the event ends, claiming also creates an ordinary pending event invitation after rechecking the organizer's authority to invite. An event that has already started may still be invited to under these ordinary rules. The link does not automatically accept participation or bypass checkout.

At or after the event's end time, or once a Speed meeting session is completed, claiming only links the external participant ID to the account. Registration can finish normally, with no new invitation or event-membership change. Existing memberships are retained. The account-linking result remains available to the integration even though the event has ended.

6.3 Read completed encounters

GET `/events/{externalId}/encounters?offset=0&limit=1000` requires a completed Speed meeting session. Each unordered pair appears once across the frozen, actual memberships of completed table rounds. A shared event alone is insufficient; ratings and friendship status do not affect this export.

```
{
  "id": "1066b61b-c0ac-43d5-ab28-f588f5a162f6",
  "externalId": "55fe8563-f360-48cf-af0e-7cfb0404ee84",
  "revision": 8,
  "pairs": [{
    "first": {"id": "guest-001", "externalId": "a5886430-3035-3f6a-a7b2-03b04c3f5911"},
    "second": {"id": "guest-002", "externalId": "4ce757f6-aec7-3cf2-aed5-9eec40bbb0fc"}
  }],
  "total": 1,
  "nextOffset": null
}
```

A participant's `id` is the caller's linked participant ID or null. Their `externalId` is an opaque organizer-scoped identifier, not an internal user ID. No contact information is exported. Use nonnegative `offset` and `limit` 1-1000; continue with `nextOffset` until null. Keep pages from the same session `revision` together. An incomplete session returns 409; an inaccessible event returns 404.

7. Optional Image Upload

Use the same `POST /assets` or `POST /events` URL with `multipart/form-data`.

Part	Content type	Content
<code>request</code>	<code>application/json</code>	The complete batch object with its <code>items</code> array
Item UUID, for example <code>1066b61b-...</code>	<code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code> , or <code>image/gif</code>	One optional local image file for that item

Every file-part name must match an `id` in the JSON batch. At most one image may be attached to an item.

The server stores the original bytes and creates WebP variants whose maximum dimensions are 128 px, 640 px, and 1280 px. The resource points to the large variant.

No remote retrieval

Do not send an image URL. The client and this API never retrieve an image from another server; either send the file bytes in this request or omit the image.

8. Change Callback

POST `/watch` registers one callback URL for the current token/client pair. It is a push notification mechanism, not a polling or general read endpoint. MyScoutee sends `event.changed` for managed events, `asset.changed` for owned assets, and explicitly selected `event.encounters` when Speed meeting completes.

```
{
  "callbackUrl": "https://integration.example.test/hooks/myscoutee",
  "events": ["event.changed", "asset.changed"],
  "enabled": true
}
```

An empty or omitted `events` list selects `event.changed` and `asset.changed`. Add `event.encounters` explicitly before session completion to receive encounter results. Disable the registration with `{"enabled": false}`. Revoking or expiring the client key also disables delivery. Change capture only writes a durable queue record on the application request path; an elastic worker pool grows from zero idle workers to at most 100 parallel deliveries, performs callback HTTP requests independently, and can deliver to multiple client destinations in parallel.

Delivered message

```
{
  "deliveryId": "ad01bce2-70ac-4a60-8703-8f03bf105af6",
  "type": "event.changed",
  "occurredAt": "2026-09-19T22:28:46.947Z",
  "resource": {
    "id": "1066b61b-c0ac-43d5-ab28-f588f5a162f6",
    "externalId": "55fe8563-f360-48cf-af0e-7cfb0404ee84",
    "title": "Hotel evening",
    "status": "A",
    "change": {
      "kind": "participant",
      "status": "accepted",
      "role": "Member",
      "actionAt": "2026-09-19T22:28:46.919Z",
      "participant": {
        "name": "Peter",
        "age": 27,
        "gender": "man",
        "profileImage": "/media/public?key=.../large.webp"
      }
    }
  }
}
```

A normal participant Join and its later membership-state change trigger `event.changed`. The optional participant projection contains only name, age, gender, and first profile image; it contains no e-mail address or internal user ID. Event snapshots also carry `sourceLink`, `mode` and `mingleConfiguration`; snapshots may additionally include event times and accepted/pending counts or asset type, quantity and capacity.

Completed encounter callback

The `event.encounters` envelope uses the same `deliveryId`, `type`, `occurredAt` and `resource` structure. Its resource contains the encounter response above plus `offset`. Results are split into messages of at most 1000 pairs; even an empty result produces a final page. Group pages by event and revision and use offsets to assemble them, since callback delivery order is not guaranteed.

Completion is saved before publication to the durable delivery queue. Pending publication is retried after a restart, with stable delivery IDs per subscription, event, revision and offset. Receiver-side deduplication is still required for delivery retries. Register before completion; previously completed sessions remain available from the read endpoint.

Delivery rule	Contract
Success	Any HTTP 2xx response
Identity	<code>deliveryId</code> equals the <code>X-MyScoutee-Webhook-Id</code> header; receivers must deduplicate it
Retry	At most three delivery attempts in total: the initial request and two retries, approximately 5 and 30 seconds after failure. After the third failure, delivery is marked failed. Retries resend the notification; they do not repeat the event operation. Completed encounters remain available through the read endpoint.
Timeout	Connection timeout: 5 seconds. Each HTTP exchange has a 10-second deadline covering the complete response, including its body. A stalled request is cancelled and counted as a failed attempt.
Redirect	Not followed
Production URL	HTTPS with a publicly resolvable destination; localhost/private HTTP is development-only

9. Responses and Errors

HTTP/result	Meaning
200 / created	The resource was created; <code>externalId</code> contains its MyScoutee UUID.
200 / existing	The caller <code>id</code> was already mapped; the same <code>externalId</code> is returned.
200 / rejected	That item failed validation or creation; <code>error</code> contains a stable reason and <code>externalId</code> is null.
400	The batch envelope or multipart relationship is invalid.
403 / 404	Invitation is forbidden, or the requested event is unavailable to this owner.
409	The encounter session is not complete, or the event has the wrong mode.

HTTP/result	Meaning
401	The key is missing, invalid, expired, revoked, or claimed by another client UUID.

Common item errors include `id_must_be_uuid`, `duplicate_id_in_batch`, `title_required`, `subtitle_required`, `valid_start_and_end_required`, and the capacity-specific validation codes.

10. Java Client and CLI

```
./gradlew installDist

myscoutee-client configure \
  https://example.com/api/integrations/v1 \
  'msc_your_token'

myscoutee-client event <myscoutee-event-id>
myscoutee-client invites <myscoutee-event-id> guest-001 guest-002
myscoutee-client encounters <myscoutee-event-id> 0 1000
myscoutee-client assets assets.json
myscoutee-client events events.json \
  1066b61b-c0ac-43d5-ab28-f588f5a162f6=event.webp
myscoutee-client watch https://integration.example.test/hooks/myscoutee
myscoutee-client unwatch
```

SDK method	Use
<code>connect()</code>	Validate and claim the configured key.
<code>createAssets(items)</code>	Send a JSON-only asset batch.
<code>createAssets(items, images)</code>	Send assets and a map from item UUID to local image path.
<code>createEvents(items)</code>	Send a JSON-only event batch.
<code>createEvents(items, images)</code>	Send events and optional local images.
<code>watch(callbackUrl, events)</code>	Register the callback and selected event types for this client.
<code>event(externalId)</code>	Read the organizer link and format configuration for a managed event.
<code>inviteParticipants(externalId, participantIds)</code>	Create or retrieve participant links.
<code>encounters(externalId, offset, limit)</code>	Read one completed encounter page.
<code>unwatch()</code>	Disable the current client's callback.

The CLI stores the Base URL, token, and stable client UUID in `~/.myscoutee/client.properties`. The token is masked by `show-config`.

11. Security and Operational Rules

- ☐ Use HTTPS outside localhost development.
- ☐ Keep the complete key secret and use a distinct key for every connected client.
- ☐ Revoke a lost or retired key from the profile popup immediately.

- ☐ Retain both `id` and `externalId` in the external system.
- ☐ Split imports into batches of no more than 1000 items.
- ☐ Do not infer access to another user's records: the token fixes the resource owner.
- ☐ Do not include participant personal data in asset or event create requests.
- ☐ Treat invitation URLs as private bearer links. Do not place them in public logs or share one participant's link with another.
- ☐ Return a fast 2xx from the callback and process asynchronously; deduplicate every delivery ID.
- ☐ Expose callback endpoints over public HTTPS in production and reject unexpected payload types.

Normative machine-readable contract: `myscoutee-client/openapi.yaml`. Reference client and versioned fixtures: `myscoutee-client/src/`.

12. MSC-116: Access Packs and Application Queries

This source extension requires the corresponding deployed backend, Java client 1.5.0 and MCP 1.5.0. Consult authenticated `connect` for the actual installed operation list and effective access.

Pack	Meaning
Blocked	No private integration calls. Already issued OAuth tokens and queued callbacks honor blocking.
Write (default)	Supported creation, invitations and organizing writes, their necessary results and the existing managed-event workflow.
Full	Write plus permitted personal/workspace reads. This exposes more data, including messages and existing ratings, without adding decision authority.

Use the colored shield beside each API key, each manual/OAuth MCP client and the main MCP URL. The main MCP pack caps its clients; API clients are independent. The small popup tick stages its value. The parent tick saves all staged values in one revision-checked request; X discards the draft.

Full reads cover visible events, groups, chats/messages, notifications and ratings, base-profile assets, Work campaigns/history, and Community services/cases/announcements/scheduled tasks. Lists use bounded pages and opaque `nextCursor`. MCP message reads do not mark a person as having viewed a message.

Organizing writes reuse existing domain roles and lifecycle rules. New mutation receipts contain only ID, status and version. A separate Full read is needed for broad details; payload identity cannot change the actor. Use the current version supplied by the user or a prior receipt.

Personal viewing and judgment remain in the application: ratings, interest, feedback, ballots, participation responses, endorsements, taking ownership and offer decisions are not integration writes. Their original buttons remain unchanged; there is no added AI approval workflow.

```
myscoutee-client query listEvents
myscoutee-client query listCampaigns parameters.json
myscoutee-client command updateCampaign campaign-update.json campaign-id
```

The canonical `myscoutee-client/openapi.yaml` lists every operation, parameter and allowed command. MCP discovery intersects that contract with current permissions on every call. Full-to-Write removes broad reads; blocking removes private access. Anonymous help remains available.

Embedded UI, directory publication and production artifact qualification are separate follow-ups. Existing `/watch` callbacks retain their own protocol.

13. MCP Events and OAuth User Isolation

MCP 1.5.0 implements `server/discover`, `events/list`, `events/subscribe` and `events/unsubscribe` on the authenticated `/mcp/private` endpoint, using MCP protocol 2026-07-28. Both the main MCP link and the client need Full access. Anonymous help and static API-key stdio do not advertise background Events.

Each person signs in and approves their own profile through OAuth 2.1 authorization code with S256 PKCE. Shared dynamic client registration is client metadata; grants and profile permissions remain separate for each user. Publisher credentials are not reused for other people.

Event	Filter and payload
event.changed	Required resourceId: an event you organize. Status/title summary, including participation changes.
event.encounters	Required resourceId: an event you organize. Completed Mingle result revision/total; retrieve pairs with the paged encounters tool.
asset.changed	Required resourceId: an asset you own in the base profile. Status/title summary, including reservation changes.

The filter is the API resource's externalId. Callback destinations use public HTTPS on port 443. The receiver supplies a `whsec_` signing secret and must echo a signed verification challenge. Delivery uses Standard Webhooks signatures over the exact body, event ID and timestamp. Keys are encrypted with the existing deployment configuration key. The ordinary protected key backup procedure applies.

The backend reuses its durable webhook queue. Subscriptions default to a maximum one-day lifetime and can be refreshed with the same identity; shorter positive ttIMs values are honored. Secret rotation provides a two-minute overlap. There is no replay: cursor is null. Expiry, unsubscribe, revoked grant/client, access downgrade and loss of ownership or membership stop delivery. Human ratings and decisions remain in the application.

Virtual delivery threads retain the common concurrency cap. Events defaults to four callback starts per second with burst four, eight concurrent requests and two simultaneous verifications. The normal `INTEGRATION_EVENTS_REQUESTS_PER_SECOND`, `INTEGRATION_EVENTS_BURST` and `INTEGRATION_EVENTS_MAX_CONCURRENT` environment values tune these limits per process. Slow responses are limited to ten seconds; transient failures have two retries. Local throttling does not spend those retries.

Queue admission is bounded at 1000 pending Events deliveries per profile and 10000 per database, with at most 100 active subscriptions per profile. Already admitted concurrent publications may briefly exceed queue admission ceilings. Overload records a gap and reports

truncated on refresh; it does not fail the source app action. Events deliveries have seven-day retention. Rescan the plugin in the host after upgrading. Live callback, ChatGPT and load QA remain pending under MCP-EVENTS-001.